

Построение эмпирической модели производительности in-memory файловой системы

Команда: ITMO-Avito Team 7

ФИО	Роль	Зона ответственности (Что делал)	Ветка GitHub
Павел Лобанов	Куратор	Курировал	
Александра Демелева	Dev 1	Разработчик ядра (FS Architect): базовый интерфейс и наивное дерево (Approach A)	interface_and_type_A
Роман Сергеев	Dev 2	Оптимизатор ФС (FS Optimizer): реализация подходов B и C (дерево+хеш и плоское хеш-отображение)	type_B / type_C
Даниил Ефимов	Dev 3	Инженер по тестированию: параметризуемый бенчмарк и генерация многомерной сетки нагрузки	benchmark
Егор Чепасов	Dev 4	Data Scientist: построение и обучение ML-моделей, анализ качества аппроксимации	ml-models
Дарья Юрьева	Dev 5	Интегратор, программа-рекомендатель, многокритериальные стратегии, пайплайн автоматизации	recommender, testing

Оглавление

Интерфейс файловой системы	4
Асимптотика методов трех подходов	4
Параметры структуры файловой системы	4
Глубина дерева D	4
Ширина дерева W	5
Доля файлов F	5
Параметры нагрузки файловой системы	5
Доля чтения P_read	5
Доля записи P_write	5
Доля создания директорий P_mkdir	5
Доля ls - P_ls	6
Доля перемещений P_mv	6
Доля поиска P_find	6
Параметры распределения доступа к файловой системе	6

Распределение обращений Dist	6
Локальность обращений Loc	7
Параметры эксперимента	7
Количество операций Ops	7
Количество повторов Repeats	7
Реализация А: наивное N-арное дерево	7
Memory Pool	8
Path Utils	8
Реализация подходов В и С	8
1. Цели и задачи разработки	8
2. Краткий обзор требований к FS и интерфейса IFileSystem	9
3. Базовые утилиты: MemoryPool и PathUtils	9
Memory Pool (Free-list аллокатор)	9
Обоснование необходимости	9
Описание работы Free-List аллокатора	9
Утилиты PathUtils	9
Обоснование необходимости	9
Описание реализованных методов	10
4. Система оценки памяти	10
5. Подход В: Дерево + Хэш-таблица	10
6. Подход С: Плоское хэш-отображение	10
Отчёт по разработке бенчмарка	11
Эмпирическая модель производительности in-memory файловой системы	11
1. Цель	11
2. Сравниваемые подходы	11
3. Архитектура benchmark	11
4. Структурные параметры	12
5. Профили операций	12
6. Распределение доступа и локальность	12
7. Размер сетки	13
8. Метрики	13
9. Формат CSV	13
10. Оценка времени эксперимента	14
11. Влияние параметров на подходы	14
12. Вывод	14
ML-Model	14
Выбор аппроксимирующей функции	15
Проблема комбинаторного взрыва	15
а) Линейная регрессия	15
б) Полиномиальная регрессия	16
в) Дерево решений	16
г) Random Forest	16
д) Gradient Boosting	16
Реализация	17
Интерфейс взаимодействия	18
Анализ обученной модели	19
Анализ корреляции между метриками (ТЗ §8.2)	19
Качество моделей по подходам и метрикам	20
Интегральная метрика качества и корреляция ошибок	20

Программа - рекомендатель	22
Соответствие пунктам ТЗ	22
Разделение ответственности	23
Стратегии выбора и их обоснование	23
Однометричные стратегии (latency, p99, memory, throughput)	23
Взвешенные стратегии (balanced, *-critical)	23
Стратегии с ограничениями (constrained)	23
Полный список 9 стратегий выбора	23
1. Однометричные стратегии (Single-metric)	24
2. Взвешенные многокритериальные стратегии (Weighted)	24
3. Стратегия с ограничениями (Constrained)	24
Таблица весов взвешенных профилей	25
Программа-рекомендатель (§6 ТЗ)	25
Параметры CLI	26
Формат вывода	26
Почему Python, а не C++	26
Архитектура и интеграция с Dev 4	26
Структура каталога recommender/	26
Слой predict.py	27
Пайплайн воспроизведения (§8.1) и надежность	27
Демонстрация работы и ключевой вывод	27
Воспроизведение: команды запуска	28
Рекомендатель (§6 ТЗ)	28
Пайплайн (§8.1 ТЗ)	28
Ограничения и границы применимости	29
Область валидности ML-предсказаний	29
Зависимость от Dev 4	29
Выводы Dev 5	29
Итоговые выводы команды (ТЗ §8.2)	29
Проблема: невозможность полного перебора пространства параметров	29
Корреляция метрик: p99 и средняя латентность	29
Оптимальность подходов по областям пространства параметров	30
Влияние стратегии выбора (Dev 5)	30
Связка компонентов проекта	31
Тестирование	31
Как запускать	31
Общая стратегия	31
Блок 1: tests/fs_core/ — ядро файловой системы	31
Блок 2: tests/benchmark/ — инфраструктура бенчмарка	32
Блок 3: tests/recommender/	32
Что сознательно не покрыто unit-тестами	33
Связь тестов с компонентами проекта	33

Интерфейс файловой системы

Все реализации файловой системы работают через общий интерфейс `IFileSystem`. Он задаёт базовый набор операций: чтение и запись файлов, создание директорий, просмотр содержимого директории, перемещение узлов, поиск по шаблону, проверку существования пути, получение состояния отдельного узла и всей системы, а также очистку файловой системы.

Интерфейс использует абсолютные пути, начинающиеся с `/`. Для представления типа узла используется `NodeType`, который может принимать значения `File` или `Dir`. Метод `NodeState` возвращает информацию об отдельном узле: путь, тип и размер служебной памяти реализации. Метод `SystemState` возвращает общую статистику: суммарный размер данных файлов, общий размер памяти реализации, количество файлов, директорий и всех узлов

Асимптотика методов трех подходов

Операция	Подход А: N-арное дерево	Подход В: дерево + hash map путей	Подход С: плоская hash map
<code>read(path)</code>	$O(D * W)$	$O(1)$ average, фактически $O(L)$	$O(1)$ average, фактически $O(L)$
<code>write(path) / перезапись</code>	$O(D * W)$	$O(1)$ average	$O(1)$ average
<code>create file</code>	$O(D * W)$	$O(1)$ average	$O(1)$ average
<code>mkdir(path)</code>	$O(D * W)$	$O(1)$ average	$O(1)$ average
<code>ls(path)</code>	$O(D * W + K)$	$O(1 + K)$	$O(N)$
<code>mv(path)</code>	$O(D * W + W)$	$O(W + S * L)$	$O(N * L)$
<code>find(path, pattern)</code>	$O(D * W + M)$	$O(1 + M)$	$O(N)$

- D - глубина пути;
- W - среднее число детей в директории;
- N - общее число узлов в файловой системе;
- K - число элементов в директории;
- M - число узлов в поддереве, по которому идёт `find`;
- S - число узлов в перемещаемом поддереве;
- L - длина строки пути.

Параметры структуры файловой системы

Глубина дерева D

D - максимальное количество уровней вложенности от корня до листа

Чем больше глубина, тем длиннее путь и тем больше уровней нужно пройти, чтобы найти файл или директорию.

- **Подход А** сильно зависит от D , потому что путь ищется последовательным проходом по дереву. На каждом уровне нужно найти нужного ребёнка среди списка детей.
- **Подход В** меньше зависит от D , потому что полный путь можно найти через `hash map`.
- **Подход С** тоже меньше зависит от D для точного доступа, потому что путь является ключом в `hash map`. Но длинные пути увеличивают стоимость хэширования и сравнения строк.

Ширина дерева W

W - это количество дочерних элементов в типичной директории

Чем больше W , тем больше элементов нужно просматривать при поиске внутри директории.

- **Подход А** сильно замедляется при росте W , потому что каждый узел хранит список детей, и поиск нужного ребёнка может требовать просмотра большого списка.
- **Подход В** меньше зависит от W при поиске по полному пути, так как использует hash map. Но `ls` всё равно зависит от количества детей в директории.
- **Подход С** почти не зависит от W при `read` и `write`, но для `ls` ему может быть тяжело, потому что без дерева нужно искать элементы с нужным префиксом среди всех путей.

Доля файлов F

F - это доля файлов среди всех узлов файловой системы

Если F высокий, то в системе много файлов и мало директорий. Это может увеличивать нагрузку на операции `read`, `write`, поиск файлов, хранение содержимого файлов.

Если F низкий, то в системе больше директорий. Тогда чаще встречаются глубокие или ветвистые структуры, и становятся сложнее операции `mkdir`, `ls`, `find`, `mv` директорий.

- Для подхода **А** изменение F влияет на форму дерева и количество элементов в списках детей.
- Для подхода **В** рост числа файлов увеличивает размер дополнительной hash map, но сохраняет быстрый доступ по пути.
- Для подхода **С** большое число файлов увеличивает количество записей в плоской таблице, из-за чего операции по точному пути остаются быстрыми, но операции, требующие анализа иерархии, могут становиться медленнее.

Параметры нагрузки файловой системы

Доля чтения P_{read}

P_{read} - вероятность того, что очередная операция будет чтением файла

Если P_{read} высокая, основная нагрузка — это быстрый поиск файла по пути и получение его содержимого

- **А** может быть медленнее, потому что каждый раз ищет файл проходом по дереву
- **В** обычно быстрее, потому что находит файл по полному пути через hash map
- **С** тоже быстро читает файл по полному пути через hash map

Доля записи P_{write}

P_{write} - вероятность создания нового файла или перезаписи существующего.

Запись требует найти нужный путь, проверить родительскую директорию и обновить содержимое или добавить новый узел.

- **А** тратит время на поиск родительской директории в дереве.
- **В** быстро находит путь через hash map, но должен обновлять и дерево, и hash map.
- **С** быстро добавляет или обновляет запись в hash map, но ему сложнее проверять структуру директорий, если нет дополнительной информации о родителях.

Доля создания директорий P_{mkdir}

P_{mkdir} - вероятность создания новой папки

Операция похожа на создание файла: нужно найти родительскую директорию и добавить новый узел.

- **A** зависит от глубины и ширины дерева.
- **B** быстрее находит родителя через hash map, но хранит новую директорию сразу в двух структурах.
- **C** просто добавляет новый путь в hash map, но иерархия хранится неявно через строки путей.

Доля **ls** - **P_ls**

P_ls - вероятность операции получения списка содержимого директории

Чем выше **P_ls**, тем важнее, насколько удобно реализации получать детей конкретной директории

- **A** хорошо подходит для **ls**, потому что у каждой директории уже есть список детей.
- **B** тоже хорошо подходит для **ls**, потому что дерево сохраняется.
- **C** работает хуже, если нет отдельного индекса детей. Так как дерева нет, приходится искать все пути, которые являются детьми данной директории.

Доля перемещений **P_mv**

P_mv - вероятность перемещения файла или директории в другое место

Перемещение может быть простой операцией, если перемещается один файл, и дорогой, если перемещается большая директория с большим количеством вложенных файлов.

- **A** может относительно просто перенести узел дерева, изменив связи между родителем и ребёнком.
- **B** сложнее при перемещении директории: нужно обновить полные пути в hash map для всех потомков.
- **C** тоже сложен для перемещения директории: нужно найти все пути с нужным префиксом и заменить их на новые.

Доля поиска **P_find**

P_find - вероятность поиска по шаблону, например поиска файлов с определенным именем или расширением

find обычно требует обхода поддерева и проверки имён файлов

- **A** может обойти нужное поддерево, начиная с заданной директории.
- **B** также может быстро найти стартовую директорию и затем обойти её поддерево.
- **C** без дерева вынужден проверять много путей в hash map, особенно если нет отдельного индекса по префиксам.

Параметры распределения доступа к файловой системе

Распределение обращений **Dist**

Dist - это правило, по которому выбираются пути для операций. Рассматриваются два варианта: равномерное распределение *Uniform* и распределение Ципфа *Zipf*

При *Uniform* обращения равномерно распределены по разным файлам и директориям. При *Zipf* одни и те же популярные файлы или папки используются чаще остальных.

- **A** может выиграть при высокой повторяемости обращений, если часто используются близкие или неглубокие пути.

- В хорошо работает и при *Uniform*, и при *Zipf*, потому что использует hash map для быстрого доступа.
- С тоже хорошо работает для точечных операций по популярным путям, но всё равно может проигрывать на ls и find.

Локальность обращений Loc

Loc - вероятность того, что следующая операция обратится к тому же поддереву, что и предыдущая

Высокая локальность означает, что операции часто выполняются рядом: в одной директории или в одном поддереве

- А может работать лучше при высокой локальности, потому что операции часто происходят в одной части дерева.
- В тоже может получать преимущество, так как быстро находит узлы и при этом сохраняет структуру дерева.
- С меньше выигрывает от локальности для ls и find, если не хранит отдельную структуру поддеревьев.

Параметры эксперимента

Количество операций Ops

Ops - длина последовательных операций в одном запуске бенчмарка

Чем больше Ops, тем дольше идёт эксперимент, но тем стабильнее измерения. Малое количество операций может дать шумные результаты.

При большом Ops лучше проявляются реальные различия между А, В и С. Например, если много операций ls, подход С будет заметно проигрывать. Если много read, подходы В и С могут быть быстрее А.

Количество повторов Repeats

Repeats - сколько раз повторяется один и тот же эксперимент для одной комбинации параметров

Повторы нужны, чтобы усреднить случайные колебания времени выполнения.

Этот параметр не меняет саму файловую систему, но помогает честнее сравнить А, В и С. Если один запуск случайно получился слишком быстрым или медленным, повторения сгладят этот эффект

Реализация А: наивное N-арное дерево

Case_A реализован классом TreeFileSystem. Это наивная древовидная in-memory файловая система. Каждый узел дерева представлен структурой Node, которая хранит тип узла, имя, данные файла, список указателей на дочерние узлы и указатель на родителя.

Дочерние элементы директории хранятся в `std::vector<Node*>`. Сами объекты Node выделяются через `MemoryPool<Node>`, поэтому родительский узел не владеет памятью детей напрямую. Поиск дочернего элемента всё равно выполняется линейным проходом по списку детей.

Корень файловой системы хранится в поле `root_` и создаётся как директория /. Для учёта состояния системы используются счётчики `total_file_bytes_`, `file_count_` и `dir_count_`.

Основные операции работают следующим образом:

1. `ReadFile(path)` находит узел по абсолютному пути и возвращает данные, если это файл
2. `WriteToFile(path, data)` перезаписывает существующий файл или создаёт новый файл в существующей родительской директории
3. `MakeDir(path)` создаёт новую директорию, если родительская директория существует
4. `List(path)` возвращает имена всех дочерних элементов директории
5. `Move(src, dest)` переносит файл или директорию в новое место. Если `dest` — существующая директория, узел добавляется внутрь неё. Если `dest` не существует, операция работает как перемещение с переименованием
6. `Exists(path)` проверяет наличие узла по пути
7. `Clear()` удаляет всё дерево и заново создаёт пустой корень /

Поскольку структура данных не содержит индексов, поиск по пути имеет сложность примерно $O(D * N)$, где D — глубина пути, а N — среднее количество элементов в директории.

Memory Pool

В реализации А для хранения узлов используется собственный пул памяти `MemoryPool<Node>`. Узлы создаются через `node_pool_.Allocate(...)`, а при удалении дерева возвращаются в пул через `node_pool_.Deallocate(...)`.

Это позволяет уменьшить количество отдельных вызовов `new/delete` при создании большого числа файлов и директорий. Пул выделяет память чанками фиксированного размера и переиспользует освобождённые блоки через `free list`.

Из-за использования `memory pool` дочерние элементы директории хранятся как обычные указатели.

При этом владение памятью принадлежит не родительскому узлу, а пулу памяти. Очистка дерева выполняется рекурсивно в `DeleteTree`, где для каждого узла явно вызывается освобождение через пул.

При подсчёте памяти реализации учитывается память, выделенная пулом под узлы, а также размер полезных данных файлов.

Path Utils

Для работы с путями используется вспомогательный модуль `PathUtils`.

`PathUtils` отвечает за проверку корректности путей, выделение имени файла или директории, получение родительского пути, разбиение абсолютного пути на компоненты и соединение частей пути.

Файловая система поддерживает только абсолютные пути, начинающиеся с `/`.

Относительные пути, пустые пути, двойные слеша и конструкции вида `.` или `..` считаются некорректными.

Также через `PathUtils` реализована поддержка `glob`-паттернов для метода `Find`. `Glob`-шаблон преобразуется в регулярное выражение, после чего имя узла проверяется через `std::regex_match`.

Реализация подходов В и С

1. Цели и задачи разработки

Цель: разработка высокопроизводительных реализаций in-memory файловой системы (подходы В и С) для последующего сбора метрик и обучения эмпирических моделей.

Задачи:

1. Проектирование и интеграция кастомного аллокатора `MemoryPool` для оптимизации скорости создания узлов и честного подсчёта системной памяти.
2. Разработка модуля `PathUtils` для эффективного парсинга путей.
3. Реализация подхода В (Дерево + Хэш-таблица) и подхода С (Плоское хэш-отображение) по заданному интерфейсному классу `IFileSystem`.

2. Краткий обзор требований к FS и интерфейса `IFileSystem`

Ограничения файловой системы: система не является полноценной *POSIX*-совместимой. В ней отсутствует архитектурное понятие «текущей рабочей директории» (*CWD*), поэтому она поддерживает только абсолютные пути. Полный отказ от относительных переходов (`.` и `..`) и сложной нормализации путей позволяет избежать коллизий в хэш-таблицах.

Интерфейс `IFileSystem`: все подходы реализуют единый общий интерфейс `IFileSystem`, спроектированный другим участником. Это гарантирует, что бенчмарк запускает одинаковую нагрузку и взаимодействует с любой реализацией абсолютно идентично.

Операции: реализована поддержка операций чтения (`read`), записи (`write`), создания директорий (`mkdir`), листинга (`ls`), перемещения (`mv`) и поиска (`find`). Также присутствуют операции проверки (`exists`), очистки системы (`clear`) и получения состояния узла/системы.

3. Базовые утилиты: `MemoryPool` и `PathUtils`

`Memory Pool` (Free-list аллокатор)

Обоснование необходимости

1. Любой подход опирается на требование динамической аллокации структур узла файловой системы (`struct Node` или `struct FsNode`).
2. Постоянные системные вызовы при использовании оператора `new` для создания новых узлов файловой системы сильно фрагментируют память и замедляют не только непосредственную работу бенчмарка, но и подготовительный этап (создания файловой системы по заданным параметрам).

Вывод: для ускорения проведения бенчмарков требуется реализовать кастомный аллокатор, который решает данные проблемы.

Описание работы Free-List аллокатора

1. Аллокатор заранее запрашивает память у ОС большими блоками (чанками), кардинально ускоряя операции `write` и `mkdir`.
2. При удалении файла память не возвращается ОС, а добавляется в связный список свободных блоков. Это гарантирует выделение и освобождение памяти на структуру узла за $O(1)$.
3. Целенаправленно отказались от STL-совместимого аллокатора, так как пул рассчитан только на объекты одного типа с параметризуемым фиксированным размером.

Утилиты `PathUtils`

Обоснование необходимости

1. Вся логика валидации и работы со строками вынесена в статические методы для соблюдения принципа единственной ответственности (Single responsibility principle).
2. Строками с путями владеют узлы файловой системы (`struct Node` или `struct FsNode`), поэтому для избавления от лишних аллокаций в куче, замедляющих работу бенчмарка, используем `std::string_view`.

Описание реализованных методов

1. Проверка валидности путей согласно принятым ограничениям.
2. Безопасное получение родительской директории и имени узла файловой системы.
3. Разбиение абсолютного пути на компоненты и соединение частей пути.
4. Поддержка пользовательских glob-паттернов через трансляцию в эквивалентное регулярное выражение и последующую проверку через `std::regex_match`.

4. Система оценки памяти

Проблема: стандартный подход подсчёта через `sizeof` показывает только «полезную» память, игнорируя реальные накладные расходы и фрагментацию чанков.

Решение: разделение зон ответственности — пул памяти отслеживает выделенную сырую системную память (чанки), а класс файловой системы — объём полезных данных файлов.

Формулы расчёта `size_t total_size_bytes`:

`total_size_bytes` = Объём чанков пула + вес полезных данных + эвристический оверхед хэш-таблицы `std::unordered_map` (массив бакетов из указателей и связанные цепочки из узлов карты)

5. Подход В: Дерево + Хэш-таблица

Структура: основная структура остаётся N-арным деревом, но дополняется хэш-таблицей

`path_index_ = std::unordered_map<std::string_view, FsNode*>`

отображающей абсолютный путь на узел.

Вспомогательные методы:

1. Внедрены приватные методы `GetNode`, `GetFile`, `GetDirectory`, `GetParentDirectory` и фабричный `CreateNewNode` для инкапсуляции поиска по хэш-таблице и валидации типов, что избавило код от бойлерплейта.
2. Внедрены методы инициализации и очистки файловой системы `InitRoot` и `DestroyTree`.
3. Добавлен вспомогательный метод рекурсивного обновления путей при перемещении директории `UpdatePathsRecursive`.

Асимптотика и компромиссы:

1. **Плюсы:** точечный доступ (`read`, `write`, `mkdir`, `exists`) работает за $O(1)$.
2. **Минусы:** операция перемещения (`mv`) директории имеет сложность $O(N)$, так как требует рекурсивного обхода поддерева для обновления полных путей и перестроения ключей в карте. Также хранение полных путей увеличивает потребление ОЗУ.

6. Подход С: Плоское хэш-отображение

Структура: полный отказ от древовидной структуры. Узлы не хранят указатели на родителей и списки детей. Вся файловая система эмулируется префиксами путей внутри единой хэш-таблицы.

`path_index_ = std::unordered_map<std::string_view, FsNode*>`

Вспомогательные методы:

1. Внедрены приватные методы `GetNode`, `GetFile`, `GetDirectory`, `GetParentDirectory` и фабричный `CreateNewNode` для инкапсуляции поиска по хэш-таблице и валидации типов, что избавило код от бойлерплейта.
2. Внедрён метод инициализации файловой системы `InitRoot`, а очистка автоматическая в силу RAII классов хэш-таблицы и пула.

Асимптотика и компромиссы:

1. **Плюсы:** экстремальная экономия памяти и точечный доступ (`read`, `write`, `mkdir`, `exists`) за $O(1)$.
2. **Минусы:** катастрофическое падение производительности для операций `ls`, `mv` и `find`. Из-за отсутствия связей между узлами эти методы требуют полного сканирования всей хэш-таблицы за $O(N)$ для поиска совпадений по строковым префиксам.

Отчёт по разработке бенчмарка

Эмпирическая модель производительности in-memory файловой системы

1. Цель

Цель benchmark-части измерить производительность трёх реализаций in-memory файловой системы при разных параметрах структуры, профиля операций и распределения доступа.

Бенчмарк должен:

- генерировать начальную файловую структуру;
- запускать одинаковую нагрузку на подходах A, B и C;
- измерять задержку, p99, throughput и память;
- сохранять результаты в CSV для дальнейшего обучения модели.

2. Сравниваемые подходы

Подход	Структура	Идея
A	Наивное дерево	Директории хранят дочерние узлы списком
B	Дерево + hash map	Дерево дополняется таблицей абсолютных путей
C	Плоская hash map	Все узлы хранятся в таблице по абсолютному пути

Все реализации используют общий интерфейс `IFileSystem`, поэтому benchmark запускает один и тот же набор операций для всех подходов.

3. Архитектура benchmark

Компонент	Назначение
WorkloadConfig	Параметры одной конфигурации нагрузки
WorkloadGenerator	Генерация setup- и measured-операций
BenchmarkRunner	Запуск операций и измерение метрик
CsvWriter	Запись результатов в CSV
main.cpp	Обход сетки и запуск экспериментов

Генерация разделена на две фазы:

- setup - создаёт начальную файловую систему и не измеряется;
- measured содержит измеряемые операции `read`, `write`, `mkdir`, `ls`, `mv`, `find`.

Такое разделение нужно, чтобы итоговые метрики отражали именно работу файловой системы под нагрузкой, а не стоимость начального построения структуры.

4. Структурные параметры

Параметр	Значения	Смысл
D	2, 5, 10, 15	Глубина дерева
W	10, 30, 100, 300	Ширина дерева
F	0.3, 0.6, 0.95	Коэффициент файловости

Значения выбраны по логарифмическому принципу, так как глубина и ширина дерева могут меняться на порядки. влияет на длину путей и число уровней обхода. Для подхода A большая глубина может увеличивать стоимость поиска по дереву. Для B и C доступ по полному пути быстрее, но глубина всё равно влияет на длину строковых путей и операции обхода. W влияет на число объектов внутри директорий. Для A широкие директории могут быть дорогими, так как дети хранятся списком. Для B lookup по полному пути ускоряется hash map, но обходы и перемещения всё равно зависят от размера структуры. Для C операции по полному пути быстрые, но ls и find могут требовать просмотра большого числа путей. F в реализации используется как коэффициент файловости: вероятность создать файл вместо директории. Значения 0.3, 0.6, 0.95 моделируют структуры с большим числом директорий, смешанные структуры и структуры с преобладанием файлов. Строгая трактовка F как доли от полного W-арного дерева не использовалась, так как число потенциальных узлов растёт как W^D . Для рекомендованных D и W это сделало бы setup-фазу слишком большой по времени и памяти.

5. Профили операций

Профиль это вектор вероятностей операций:

Профиль	read	write	mkdir	ls	mv	find
build_system	0.45	0.30	0.10	0.05	0.05	0.05
file_manager	0.20	0.10	0.15	0.30	0.20	0.05
repo_server	0.60	0.20	0.05	0.05	0.03	0.07
backup	0.55	0.05	0.05	0.20	0.02	0.13
refactoring	0.20	0.15	0.10	0.10	0.35	0.10
static_web	0.85	0.03	0.01	0.03	0.01	0.07

Выбраны 6 прикладных сценариев:

- build_system много чтения исходников и записи артефактов;
- file_manager частое чтение и умеренная запись;
- repo_server просмотр директорий, создание папок и перемещения;
- backup - чтение и обход дерева;
- refactoring - много перемещений и изменений файлов;
- static_web почти read-only нагрузка.

Такой набор покрывает read-heavy, write-heavy, metadata-heavy, scan-heavy и move-heavy режимы.

6. Распределение доступа и локальность

Dist	zipf_s	locality	Смысл
------	--------	----------	-------

uniform	1.5	0.0	Равномерный доступ
zipf	1.5	0.3	Умеренно неравномерный доступ
zipf	2.0	0.8	Сильная концентрация обращений

При uniform все объекты выбираются примерно с одинаковой вероятностью. Параметр zipf_s в этом режиме не используется и хранится только для единого формата CSV. При zipf часть объектов выбирается чаще остальных. Чем больше zipf_s, тем сильнее концентрация обращений к популярным объектам. locality задаёт вероятность обращения к той же области дерева, что и в предыдущей операции. Большая локальность моделирует сценарии, где пользователь или сервис активно работает с одним поддеревом. Эти параметры важны, потому что подходы по-разному реагируют на характер доступа. Например, В и С быстрее находят узлы по полному пути, а А сильнее зависит от обхода дерева.

7. Размер сетки

Размер сетки: 864 То есть используется:

- 6 профилей операций;
- 4 значения D;
- 4 значения W;
- 3 значения F;
- 3 варианта распределения доступа.

Каждая конфигурация запускается на трёх подходах: 2592 Итоговый CSV содержит 2592 агрегированные строки. При Ops 5000 и Repeats = 7 выполняется: 18144 измеряемых запусков. Общее число измеряемых операций: 90,720,000

8. Метрики

Метрика	Описание
avg_latency_us	Средняя задержка операции в микросекундах
p99_latency_us	99-й перцентиль задержки
throughput_ops_sec	Количество операций в секунду
memory_bytes	Оценка потребления памяти

Latency считается отдельно для каждой measured-операции через `std::chrono::steady_clock`. Setup-фаза в метрике не входит. Также сохраняются стандартные отклонения:

- avg_latency_us_std;
- p99_latency_us_std;
- throughput_ops_sec_std;
- memory_bytes_std.

Они показывают разброс результатов между повторами.

9. Формат CSV

Каждая строка CSV соответствует одной конфигурации и одному подходу. В строке сохраняются:

- approach;
- D, W, F;
- profile;
- p_read, p_write, p_mkdir, p_ls, p_mv, p_find;

- dist, zipf_s, locality;
- ops, repeats;
- средние значения метрик;
- стандартные отклонения.

Такой формат подходит для обучения модели, так как содержит входные параметры и измеренные целевые значения.

10. Оценка времени эксперимента

Финальные параметры:

Ops	5000
Repeats	7

Перед финальным запуском был выполнен пилотный прогон с меньшими значениями Ops и Repeats. По нему оценивалось полное время эксперимента. При переходе от Ops = 200, Repeats = 1 к Ops = 5000, Repeats = 7 масштабный коэффициент равен: 175. То есть финальный запуск ожидался примерно в 175 раз дольше пилотного. Оценка является приближённой, потому что время не всегда растёт строго линейно. При большем числе операций файловая система может увеличиваться из-за write и mkdir, а операции find и list могут становиться дороже. Ops = 5000 выбрано для устойчивого расчёта avg_latency_us и p99_latency_us. При 5000 операциях p99 оценивается примерно по верхним 50 операциям серии. Repeats = 7 выбран в соответствии с рекомендацией использовать 5-10 повторов. Повторы нужны для расчёта средних значений и стандартных отклонений.

11. Влияние параметров на подходы

Подход А чувствителен к глубине и ширине дерева, так как поиск выполняется через обход узлов. При больших D и W ожидается рост задержек. Подход В быстрее выполняет доступ по полному пути за счёт hash map, но использует больше памяти, так как хранит и дерево, и таблицу путей. Подход С также быстро обращается к узлам по абсолютному пути, но операции и find могут быть дороже, потому что требуют анализа префиксов путей. Поэтому один и тот же workload может быть выгоден для одного подхода и невыгоден для другого.

12. Вывод

Разработанный benchmark:

- запускает три подхода А, В и С;
- использует параметризуемую сетку;
- поддерживает прикладные профили операций;
- учитывает распределение доступа и локальность;
- измеряет latency, p99, throughput и memory;
- агрегирует результаты по нескольким повторам;
- сохраняет CSV для анализа и обучения модели.

Полученный датасет можно использовать для построения эмпирической модели производительности и выбора подходящей реализации файловой системы.

ML-Model

Цель ML-части проекта — по параметрам структуры in-memory файловой системы и профилю нагрузки предсказать производительность трех реализаций файловой системы: А, В и С. После получения прогнозов программа должна выбрать наиболее подходящую реализацию по заданной стратегии.

Модель предсказывает четыре метрики:

- *avg_latency_us* — средняя задержка операции в микросекундах;
- *p99_latency_us* — 99-й перцентиль задержки в микросекундах;
- *memory_bytes* — потребление памяти в байтах;
- *throughput_ops_sec* — пропускная способность в операциях в секунду.

Выбор аппроксимирующей функции

Проблема комбинаторного взрыва

Пространство входных параметров является многомерным. Для каждой точки необходимо выбрать глубину, ширину и заполнение дерева, профиль операций, распределение доступа, локальность и один из трех подходов. Даже текущая ограниченная сетка содержит 2592 экспериментальные точки

Каждая точка измерялась семь раз по 5000 операций. Поэтому для текущих данных было выполнено 90720000 операций

Особенно быстро пространство растет при переборе вероятностей операций. Если разделить интервал вероятности шагом 0.1, то для шести вероятностей, сумма которых равна единице, существует 3003 допустимых профиля. Тогда получается 3 378 375 точек

При семи повторах по 5000 операций это более 118 миллиардов операций. Добавление новых значений параметров, размеров серии, повторов и вариантов распределения увеличивает объем еще сильнее.

Поэтому практический подход состоит из двух этапов:

1. Выполнить реальные измерения только в заранее выбранных узлах сетки.
2. Построить аппроксимирующую модель, которая восстанавливает значения метрик между измеренными точками.

Для данной задачи это единственный практичный способ покрыть многомерное пространство параметров за разумное время.

Почему выбранная сетка разумна (связь с бенчмарком Dev 3): полный перебор вероятностей операций невозможен, поэтому используются **6 прикладных профилей** (*build_system*, *file_manager*, *repo_server*, ...). Параметры *D*, *W*, *F* взяты с **логарифмическим шагом** (2–15, 10–300), чтобы покрыть порядки величины без экспоненциального *setup*. Итого 864 конфигурации нагрузки × 3 подхода — компромисс между полнотой и 90 млн измеряемых операций (при *ops*=5000, *repeats*=7). ML-модель интерполирует **между** узлами этой сетки.

Существует множество вариантов аппроксимирующих функций, рассмотрим наиболее популярные.

а) Линейная регрессия

“**Линейная регрессия в машинном обучении** — это алгоритм обучения с учителем, который моделирует **линейные зависимости** между входными переменными (признаками) и выходной переменной (целевым значением). В основе модели лежит предположение, что зависимость между данными можно представить в виде прямой линии (или гиперплоскости в многомерном пространстве).”

Для нашей задачи обычная линейная регрессия слишком слабая. Производительность файловой системы вряд ли зависит от параметров строго линейно.

б) Полиномиальная регрессия

“**Полиномиальная регрессия** — это метод в машинном обучении, который расширяет классическую линейную регрессию, позволяя моделировать нелинейные зависимости между признаками и целевой переменной. Вместо прямой линии она строит кривую, чтобы лучше соответствовать данным.”

Количество признаков быстро растёт. Если данных мало, модель легко переобучается. Полином высокой степени может хорошо запомнить тренировочные данные, но плохо работать на новых точках.

Для нашей задачи можно попробовать полином степени 2, но не выше, если данных не очень много.

в) Дерево решений

“**Дерево решений** (дерево принятия решений) — это иерархический алгоритм машинного обучения, который последовательно разделяет данные на группы на основе значений признаков. Цель — создать модель, которая предсказывает значение целевой переменной путём обучения простым правилам принятия решений, выведенным из характеристик данных.

Алгоритм основан на правиле: «Если условие, то ожидаемый результат»”

Одно дерево часто нестабильно и может переобучаться. Поэтому лучше использовать не одно дерево, а ансамбль деревьев.

г) Random Forest

“**Random Forest** («случайный лес») — **ансамблевый алгоритм машинного обучения**, основанный на построении множества деревьев решений. Ключевая идея — комбинация нескольких простых моделей (деревьев) обычно даёт более точный результат, чем одна сложная модель.”

Он хорошо работает с нелинейными зависимостями, порогами и взаимодействиями признаков. При этом не требует огромного количества данных и обычно работает стабильно даже на табличных данных среднего размера.

д) Gradient Boosting

“**Градиентный бустинг (Gradient Boosting)** — это мощный метод машинного обучения, который строит сложную модель последовательно, объединяя множество простых «слабых» алгоритмов (чаще всего решающих деревьев) в одну сильную. Ключевая идея в том, что каждая новая модель учится исправлять ошибки тех, что были построены раньше.”

Обычно даёт высокое качество. Хорошо моделирует нелинейности и пороги. Но требует более аккуратной настройки, чем **Random Forest**. Может переобучиться, если данных мало. Интерпретируемость хуже, чем у линейной модели.

Критерий	Линейная регрессия	Полиномиальная регрессия	Дерево решений	Random Forest	Gradient Boosting
Форма зависимостей	Линейная	Нелинейная (гладкая)	Нелинейная, с порогами	Нелинейная, с сглаживанием порогов	Нелинейная, с порогами
Объём данных	Малый	Умеренный	Средний	Средний	Большой

Интерпретируемость	Высокая	Средняя	Высокая (для небольших деревьев)	Низкая	Очень низкая
Вычислительные ресурсы	Низкие	Умеренные	Низкие – умеренные	Низкие – умеренные	Высокие
Корреляции между метриками	Чувствительна (нужна регуляризация)	Чувствительна	Менее чувствительна	Устойчива	Устойчива

В итоге, выбор пал на **Random Forest**, так как он самый оптимальный и эффективный для нашей задачи.

Реализация

Начинается реализация со считывания и обработки входных данных.

В `ModelLearner.py` и `BestParamsSearcher.py` выполняется одинаковая подготовка данных.

Название подхода переводится в число:

```
“case_a -> 0
case_b -> 1
case_c -> 2”
```

Распределение доступа также кодируется числом:

```
“uniform -> 0
zipf -> 1”
```

После кодирования удаляются строки с отсутствующими значениями. Затем формируются таблица признаков X и таблица целевых значений y .

Данные делятся на обучающую и тестовую выборки в отношении 80/20:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

В качестве основного алгоритма используется `RandomForestRegressor`.

Для каждой целевой метрики обучается отдельная модель:

1. модель средней задержки
2. модель р99 задержки
3. модель потребления памяти
4. модель пропускной способности

Подход А, В или С не имеет отдельной модели. Он передаётся в модель как входной признак `approach`. Поэтому для получения результатов по всем трём подходам одна и та же группа моделей вызывается три раза с кодами 0, 1 и 2.

Основной проблемой обучения модели является подбор оптимальных параметров. Поэтому был написан скрипт для подбора параметров `BestParamsSearcher.py` с помощью `GridSearchCV`. Поиск выполняется отдельно для каждой целевой метрики.

Проверяются следующие значения:

```
{
    "n_estimators": [100, 200, 300],
    "max_depth": [None, 10, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": [None, "sqrt", "log2"],
}
```

Для каждой комбинации выполняется пятикратная кросс-валидация. Критерием выбора является минимальный RMSE.

Найденные параметры сохраняются в `best_params.json`.

Итоговое обучение выполняется в `ModelLearner.py`.

Последовательность работы:

1. Загружается `results.csv`.
2. Категориальные значения `approach` и `dist` преобразуются в числа.
3. Данные делятся на обучающую и тестовую части.
4. Из `best_params.json` загружаются лучшие параметры.
5. Для каждой из четырёх метрик создаётся отдельная модель.
6. Модели обучаются на обучающей выборке.
7. На тестовой выборке создаётся таблица предсказаний `y_pred`.
8. Вычисляются общие R^2 , MAE и RMSE.
9. Модели и служебная информация сохраняются в `model.pkl`.

Файл `model.pkl` содержит словарь:

```
{
    "models": models,
    "feature_columns": feature_columns,
    "target_columns": target_columns,
    "log_transformed_targets": [...],
    "best_params": best_params,
}
```

Интерфейс взаимодействия

Пользовательский интерфейс реализован в `Interface.py`. Главная функция называется `getPredictions`.

Она принимает:

1. `D, W, F` — параметры структуры;
2. `p_read, p_write, p_mkdir, p_ls, p_mv, p_find` — вероятности операций;
3. `dist` — `uniform` или `zipf`;
4. `zipf_s` — параметр распределения Ципфа;
5. `locality` — локальность обращений;
6. `strategy` — стратегия выбора подхода.

Если `model.pkl` отсутствует, `Interface.py` запускает `ModelLearner.py`, после чего использует созданную модель. Пути строятся относительно расположения Python-файлов, поэтому вызов не зависит от текущей рабочей директории.

Для каждого входного набора функция формирует три строки признаков:

1. approach = 0 для A
2. approach = 1 для B
3. approach = 2 для C

После этого все четыре модели предсказывают вектор метрик для каждого подхода.

Поддерживаются пять стратегий.

latency

Выбирается подход с минимальной средней задержкой avg_latency_us.

p99

Выбирается подход с минимальным значением p99_latency_us.

memory

Выбирается подход с минимальным потреблением памяти memory_bytes.

throughput

Выбирается подход с максимальной пропускной способностью throughput_ops_sec.

balanced

Комбинированная стратегия учитывает все четыре метрики с весами:

avg_latency_us — 0.4
p99_latency_us — 0.2
memory_bytes — 0.2
throughput_ops_sec — 0.2

Сначала каждая метрика нормируется относительно предсказаний A, B и C. Для задержки, p99 и памяти меньшее значение считается лучшим. Для throughput большее значение считается лучшим. Затем вычисляется взвешенная сумма нормированных ошибок. Выбирается подход с минимальным balanced_score.

В результате для balanced дополнительно возвращается strategy_scores, позволяющий увидеть оценку каждого подхода.

Анализ обученной модели

Анализ корреляции между метриками (ТЗ §8.2)

Исследуемый вопрос: является ли p99_latency_us линейной функцией от avg_latency_us, или зависимость сложнее?

Для анализа были рассчитаны коэффициенты корреляции Пирсона и Спирмена. Pearson показывает, насколько зависимость похожа на линейную. Spearman показывает, есть ли монотонная зависимость, даже если она нелинейная.

Основные результаты:

Пара метрик	Pearson	Spearman	Вывод
avg latency и p99	0.0639	0.9517	Сильная монотонная, но не линейная зависимость
avg latency и throughput	-0.0305	-0.9998	Почти строгая обратная зависимость
p99 и throughput	-0.3875	-0.9511	При росте p99 throughput обычно падает

p99 и memory	0.5917	0.3725	Умеренная связь
memory и throughput	-0.1329	-0.4011	Слабая или умеренная обратная связь

Главный результат связан с avg_latency_us и p99_latency_us. Если бы p99 был простой линейной функцией от средней латентности, его можно было бы описать формулой:

$$p99_latency_us = a * avg_latency_us + b$$

В таком случае коэффициент Pearson был бы близок к 1. Но в реальных данных он равен только 0.0639. Это означает, что прямая линия почти не описывает зависимость между средней задержкой и p99.

При этом Spearman равен 0.9517. Значит, связь между метриками всё-таки сильная: при росте средней латентности p99 обычно тоже растёт. Но растёт он не по линейному закону. Форма зависимости более сложная: на неё влияют тип операций, глубина и ширина дерева, распределение обращений и редкие тяжёлые операции.

Также видно, что средняя латентность и throughput почти идеально связаны по Spearman: -0.9998. Это логично: чем больше времени занимает одна операция, тем меньше операций система успевает выполнить за секунду. Память связана с задержками слабее, потому что она сильнее зависит от выбранной структуры данных и размера файловой системы.

Вывод: p99 не является линейной функцией от средней латентности. Между ними есть сильная монотонная зависимость, но она нелинейная и чувствительная к редким медленным операциям. Поэтому p99 нужно предсказывать отдельной моделью, а не вычислять напрямую из avg_latency_us.

Качество моделей по подходам и метрикам

Качество было рассчитано на той же тестовой выборке 20%, которая формируется в ModelLearner.py с random_state=42. Для каждого подхода и каждой метрики рассчитаны R^2 , MAE и RMSE.

Подход	Метрика	R^2	MAE	RMSE
A	avg latency	0.9918	1.1832	2.6382
A	p99 latency	0.9946	20.8384	49.4080
A	memory	0.9997	827.8958	2393.6639
A	throughput	0.9862	27917.4287	58124.4792
B	avg latency	0.9612	1.8994	4.7076
B	p99 latency	0.9980	13.7487	24.1759
B	memory	0.9996	4645.2497	7155.9053
B	throughput	0.9759	50089.4447	122349.0780
C	avg latency	0.9108	4.0615	10.6542
C	p99 latency	0.9950	14.3267	42.3931
C	memory	0.9996	4343.2979	6409.3296
C	throughput	0.9610	20967.0900	68185.6874

Интегральная метрика качества и корреляция ошибок

Так как модель одновременно предсказывает четыре выходных переменные с разными единицами измерения, обычные MAE и RMSE по всем столбцам не очень удобны для

итоговой оценки. Ошибка в байтах или операциях в секунду численно намного больше ошибки в микросекундах, поэтому такие метрики могут быть смещены в сторону крупных по масштабу величин.

Для более честной общей оценки можно использовать среднюю нормированную абсолютную ошибку. Для каждой метрики ошибка нормируется на диапазон значений этой метрики на тестовой выборке:

```
normalized_error = |y_true - y_pred| / (max(y_true) - min(y_true))
```

После этого ошибки усредняются сначала по объектам тестовой выборки, а затем по четырём выходным переменным:

```
integral_error = mean(NMAE_avg_latency,  
                      NMAE_p99_latency,  
                      NMAE_memory,  
                      NMAE_throughput)
```

Полученные значения:

Метрика	Нормированная MAE
avg latency	0.0096
p99 latency	0.0053
memory	0.0018
throughput	0.0078
среднее по всем метрикам	0.0061

Это означает, что в среднем абсолютная ошибка модели составляет примерно 0.61% от диапазона соответствующей метрики на тестовой выборке.

Также была проверена корреляция между ошибками предсказания разных метрик. Для этого рассматривались абсолютные нормированные ошибки.

Корреляция Пирсона между абсолютными нормированными ошибками:

Пара ошибок	Pearson
avg latency и p99	0.4012
avg latency и memory	0.0681
avg latency и throughput	-0.1194
p99 и memory	0.0131
p99 и throughput	-0.1239
memory и throughput	-0.0301

Корреляция Спирмена между абсолютными нормированными ошибками:

Пара ошибок	Spearman
avg latency и p99	0.6325
avg latency и memory	0.1994
avg latency и throughput	-0.4081

p99 и memory	0.0388
p99 и throughput	-0.4750
memory и throughput	-0.0741

Главный вывод: ошибки предсказания средней задержки и p99 действительно связаны. Однако связь не является абсолютной: Pearson равен 0.4012, а Spearman равен 0.6325. Значит, модель не всегда одновременно ошибается в обеих метриках.

Ошибки памяти почти не коррелируют с ошибками задержек и throughput. Это показывает, что модель предсказывает память по другим закономерностям. Ошибки throughput имеют отрицательную корреляцию с ошибками latency и p99 по Спирмену. Это связано с тем, что throughput почти монотонно зависит от задержки, но ошибка в одной метрике не обязана численно повторяться в другой.

Таким образом, анализ ошибок подтверждает, что отдельные модели для разных target оправданы. Метрики связаны между собой, особенно avg_latency_us, p99_latency_us и throughput_ops_sec, но ошибки по ним не полностью дублируются.

В результате был построен полный ML-пайплайн: подготовка экспериментальных данных, подбор гиперпараметров, обучение регрессионных моделей, оценка качества, сохранение моделей и программный интерфейс-рекомендатель. Пользователь передаёт параметры файловой системы и нагрузки, получает прогнозы четырёх метрик для подходов А, В и С и итоговую рекомендацию в соответствии с выбранной стратегией.

Программа - рекомендатель

В рамках архитектуры проекта уже реализованы in-memory файловые системы (Dev 1–2), бенчмаркинг и замеры производительности (Dev 3), а также обучение ML-модели для предсказания метрик (Dev 4).

Зона ответственности Dev 5 закрывает финальный этап — **принятие бизнес-решения**. Один и тот же прогноз производительности (латентность, память, пропускная способность) можно интерпретировать совершенно по-разному в зависимости от того, что важнее заказчику: жесткие лимиты по RAM, скорость ответа или пропускная способность.

Суть работы модуля можно выразить одной фразой: **ML предсказывает метрики → рекомендатель по выбранной стратегии выбирает подход → пайплайн позволяет воспроизвести данные и модель офлайн**.

Важно отметить: рекомендатель не меряет производительность и не обучает модель — он является инструментом аналитики поверх уже предсказанных чисел. Это позволило нам четко разделить предсказание (Dev 4) и бизнес-логику выбора (Dev 5).

Соответствие пунктам ТЗ

Пункт ТЗ	Требование	Реализация Dev 5
§6	Параметры нагрузки и стратегия на входе	recommend.py (argparse)
§6	Метрики А/В/С + стратегия + рекомендация на выходе	format_report(), флаг --json
§6	Без запуска бенчмарка, только арифметика	predict.py + strategies.py
§6	Несколько стратегий, в т.ч. с ограничениями	strategies.py: 9 стратегий

§8.1	Пайплайн: сборка → прогон → CSV → обучение	pipeline.py
§8.2 п.8	Демонстрация на примерах	раздел «Демонстрация»
§8.2 п.9	Смена стратегии меняет выбор подхода	таблица стратегий в демо

Разделение ответственности

Каталоги других разработчиков (benchmark/, case_A/B/C/, utils/, ML-Model/) **не изменяются**. Вся логика Dev 5 — в recommender/. Интеграция только через публичные точки входа:

1. C++-бенчмарк — сборка benchmark_app через CMake;
2. ML — импорт ML-Model/Interface.py, поле result["predictions"]; встроенный recommended_approach Dev 4 игнорируется.

Стратегии выбора и их обоснование

Это ключевой алгоритмический блок программы-рекомендателя. Поскольку оптимального подхода “в вакууме” не существует, система поддерживает три класса стратегий, отражающих разные инженерные приоритеты.

Однометричные стратегии (latency, p99, memory, throughput)

Применяются, когда в системе есть жестко доминирующий приоритет. Выбирается один критерий, и по нему определяется абсолютный победитель (например, минимальное потребление RAM для embedded-устройств).

Взвешенные стратегии (balanced, *-critical)

Используются, когда необходимо найти компромисс между метриками. Поскольку метрики имеют разную размерность (микросекунды, байты, операции/сек), применяется метод **Min-Max нормализации** штрафов (от 0 до 1) исключительно по трем рассматриваемым подходам (A, B, C). Итоговый выбор делается на основе взвешенной суммы штрафов. Это позволяет избежать ситуации, когда подход выигрывает миллисекунду скорости, но требует в 10 раз больше памяти.

Стратегии с ограничениями (constrained)

Наиболее приближенный к реальным условиям сценарий (SLA-driven). Алгоритм работает в два этапа:

1. Отсекаются подходы, не проходящие заданные лимиты (например, p99_latency_us <= 5000 мкс, то есть 5 мс).
2. Среди оставшихся выбирается лучший по целевой метрике (--objective).

Если ограничения невыполнимы ни для одного подхода, система не падает с ошибкой, а вычисляет суммарное относительное нарушение и предлагает “наименьшее из зол”, явно предупреждая пользователя.

Полный список 9 стратегий выбора

В то время как модуль Dev 4 сфокусирован на аппроксимации и вычислении векторов метрик, модуль Dev 5 существенно расширяет логику принятия решений. Dev 4 даёт predictions; Dev 5 добавляет профили *-critical и constrained, увеличивая общее число доступных стратегий с 5 до 9.

Все 9 стратегий разделены на три фундаментальных класса в зависимости от математической модели выбора и операционных задач:

1. Однометричные стратегии (Single-metric)

Данный класс включает 4 стратегии, каждая из которых ориентирована на оптимизацию строго одного целевого показателя. Они используются в сценариях с бескомпромиссными техническими требованиями к инфраструктуре:

- **latency** — выбор подхода с минимальной средней задержкой выполнения операций. Оптимально для интерактивных систем с высокой частотой запросов, где критичен средний темп ответа на действия пользователя.
- **p99** — минимизация 99-го перцентиля задержки. Стратегия нацелена на подавление «длинных хвостов» распределения, возникающих из-за редких блокировок, реаллокаций или коллизий в структурах данных. Необходима для систем жесткого реального времени и строгого соблюдения SLA.
- **memory** — выбор архитектуры с минимальным объемом потребляемой памяти реализации. Критически важна для embedded-окружений, микроконтроллеров или сценариев с высокой плотностью контейнеризации, где превышение лимита RAM приводит к аварийному завершению процесса операционной системой (OOM-killer).
- **throughput** — максимизация пропускной способности (количество операций в секунду). Применяется в пакетных процессорах, системах массовой индексации данных и bulk-нагрузках, где задержка отдельного запроса вторична по сравнению с общим объемом обработанной информации в единицу времени.

2. Взвешенные многокритериальные стратегии (Weighted)

Этот класс содержит 4 стратегии, предназначенные для поиска оптимального компромисса (Парето-эффективных точек) в условиях, когда ни один KPI не является абсолютно доминирующим. Математическая модель основана на локальной Min-Max нормализации метрик по трем подходам (A, B, C) для приведения их к единой шкале относительных штрафов $[0, 1]$, после чего вычисляется взвешенная сумма штрафов:

- **balanced** — базовый сбалансированный профиль, распределяющий веса равномерно между скоростью работы и памятью. Позволяет найти наиболее стабильный архитектурный компромисс и избежать катастрофической деградации второстепенных параметров ФС при общей универсальной нагрузке.
- **latency-critical** — профиль, в котором веса сильно смещены в сторону avg_latency_us (0.50) и p99_latency_us (0.30). При этом штрафы за избыточное выделение памяти или потерю общей пропускной способности все еще участвуют в минимизации, страхуя систему от экстремально неэффективных по ресурсам решений.
- **memory-critical** — стратегия, где вес штрафа за память является определяющим (0.60). Однако при равенстве или высокой близости показателей памяти среди нескольких подходов, выбор автоматически будет сделан в пользу более быстрого и производительного алгоритма.
- **throughput-critical** — конфигурация, оптимизирующая валовую производительность как ключевой параметр (высокий вес 0.60 на throughput_ops_sec), с сохранением математического контроля над тем, чтобы задержки и потребление памяти оставались в рамках допустимых относительных отклонений.

3. Стратегия с ограничениями (Constrained)

Девятая стратегия — **constrained** — представляет собой уникальный двухэтапный механизм оптимизации, симулирующий реальный инженерный процесс проектирования систем на основе заданных жестких бизнес-требований:

1. **Фильтрация (Feasible Pool):** На основе переданных пользователем через CLI пороговых значений (например, `--max-p99`, `--max-memory`, `--min-throughput`) формируется подмножество подходов, удовлетворяющих абсолютно всем условиям одновременно.
2. **Оптимизация:** Среди подходов, успешно прошедших фильтр, выбирается наилучший по целевой метрике, заданной в параметре `--objective`.

Dev 4 в `Interface.py` поддерживает 5 стратегий выбора. Dev 5 расширяет набор до 9: добавлены профили `*-critical` и стратегия `constrained`.

Класс	Стратегия	Критерий / суть
Однометричные (4)	latency	min avg_latency_us
	p99	min p99_latency_us
	memory	min memory_bytes
	throughput	max throughput_ops_sec
Взвешенные (4)	balanced	веса 0.4 / 0.2 / 0.2 / 0.2
	latency-critical	латентность доминирует (0.5 / 0.3 / 0.1 / 0.1)
	memory-critical	память доминирует (0.6 / 0.2 / 0.1 / 0.1)
	throughput-critical	throughput доминирует (0.6 / 0.2 / 0.1 / 0.1)
С ограничениями (1)	constrained	фильтр по SLA + <code>--objective</code>

Таблица весов взвешенных профилей

Профиль	avg lat.	p99	memory	throughput
balanced	0.40	0.20	0.20	0.20
latency-critical	0.50	0.30	0.10	0.10
memory-critical	0.20	0.10	0.60	0.10
throughput-critical	0.20	0.10	0.10	0.60

Min-max нормализация выполняется **по трём подходам А/В/С** для текущей нагрузки (не по всему CSV): лучший подход по метрике получает штраф 0, худший — 1. Это относительное сравнение «здесь и сейчас» при выборе из трёх кандидатов.

Логика обработки невыполнимых ограничений: Если в процессе работы ни один из подходов не способен уложиться в жесткие критерии SLA, алгоритм переходит в режим грациозной деградации. Он рассчитывает суммарное относительное нарушение ограничений для каждого кандидата и рекомендует вариант с наименьшим совокупным отклонением от нормы, снабжая вывод диагностическим предупреждением в поле `notes`.

Программа-рекомендатель (§6 ТЗ)

Рекомендатель (`recommend.py`) представляет собой CLI-инструмент (и потенциально микросервис) для инженеров.

Входные данные: параметры нагрузки (структура дерева, профиль операций, распределение) и выбранная стратегия с ее параметрами (например, пороги для `constrained`).

Выходные данные: подробный текстовый отчет или JSON-структура, включающая таблицу предсказанных метрик по А, В, С, имя примененной стратегии, скорости и итоговую рекомендацию.

Программа обрабатывает за миллисекунды, так как выполняет только быструю арифметику поверх весов ML-модели. Это сервис принятия решений для рантайма, а не еще один тяжеловесный бенчмарк.

Параметры CLI

Группа	Параметры	Смысл
Структура	--D, --W, --F	глубина, ширина, доля файлов
Операции	--p-read ... --p-find	вероятности типов операций
Доступ	--dist, --zipf-s, --locality	uniform/zipf, локальность
Стратегия	--strategy	одна из 9 стратегий
Ограничения	--max-pr99, --max-memory, ...	только для constrained
Вывод	--json	машиночитаемый JSON

Формат вывода

Текстовый отчёт: параметры нагрузки → таблица метрик A/B/C → стратегия и скоры → строка ==> Рекомендуемый подход: X.

JSON (--json): params, predictions, strategy, recommended_approach, scores, feasible, notes.

Почему Python, а не C++

Язык	За	Против
Python	нативная интеграция с ML Dev 4; быстрая итерация над стратегиями	медленнее C++ (не критично)
C++	единый стек с ФС	вызов sklearn/joblib сложнее; не выбрано

Архитектура и интеграция с Dev 4

Поток данных:

1. Пользователь передаёт параметры и стратегию в `recommend.py`.
2. `predict.py` вызывает `getPredictions` Dev 4 и извлекает `predictions` по A/B/C.
3. `strategies.py` применяет стратегию и возвращает `Recommendation`.
4. CLI форматирует текстовый отчёт или JSON.

Формально: `params` → `predictions_{A,B,C}` → `recommended_approach`.

Структура каталога `recommender/`

Файл	Назначение
<code>strategies.py</code>	9 стратегий выбора поверх <code>predictions</code>
<code>predict.py</code>	обёртка над <code>Interface.getPredictions</code>
<code>recommend.py</code>	CLI и форматированный вывод (§6)
<code>pipeline.py</code>	пайплайн §8.1: <code>cmake</code> → <code>benchmark</code> → <code>CSV</code> → <code>train</code>
<code>requirements.txt</code>	Python-зависимости
<code>README.md</code>	документация по запуску

Тесты: `tests/recommender/` (20 unit-тестов).

Слой `predict.py`

`predict_metrics(params)` возвращает словарь предсказаний по подходам A, B, C с четырьмя метриками: `avg_latency_us`, `p99_latency_us`, `memory_bytes`, `throughput_ops_sec`.

Почему `importlib`: каталог ML-Model содержит дефис — стандартный `import` невозможен; используется `importlib.util.spec_from_file_location`.

Почему игнорируется `recommended_approach` Dev 4: вектор `predictions` не зависит от стратегии; `constrained` — зона Dev 5; стратегии не должны требовать правок чужого кода. В `getPredictions` передаётся фиксированная стратегия "latency", возвращается только `result["predictions"]`.

Архитектура модуля выстроена в три слоя: `predict.py` (адаптер) → `strategies.py` (логика) → `recommend.py` (интерфейс).

Пайплайн воспроизведения (§8.1) и надёжность

Для автоматизации полного цикла эксперимента написан скрипт `pipeline.py`. Он позволяет одной командой воспроизвести весь путь: сборка C++ проекта (CMake) → запуск бенчмарка для получения `results.csv` → (опционально) переобучение модели.

Пайплайн выступает оркестратором. Сама сетка конфигураций (D/W/F, ops, repeats) формируется на стороне бенчмарка (зона Dev 3), а пайплайн лишь автоматизирует ее прогон.

Безопасность по умолчанию: Скрипт защищает артефакты ML-слоя. По умолчанию сборка — в `recommender/build/`, CSV — в `recommender/data/`. Папка ML-Model/ перезаписывается только при явном указании флагов `--install-dataset` и `--train`, при этом старые данные бэкапятся.

Пайплайн **не дублирует** сетку экспериментов — она задаётся в `benchmark/main.cpp` (`MakeGrid()`): $6 \times 4 \times 3 \times 3 = 864$ конфигурации $\times 3$ подхода = 2592 строки CSV.

Подробное описание тестов — в разделе «Тестирование» в конце отчёта.

Демонстрация работы и ключевой вывод

Базовая нагрузка: $D = 10$, $W = 100$, $F = 0.6$; профиль операций близок к `repo_server` (`p_read=0.6`, `p_write=0.2`, ...); `dist=zipf`, `zipf_s=1.5`, `locality=0.3`.

Предсказанный вектор метрик (ML-модель Dev 4):

Подход	avg lat., мкс	p99, мкс	memory, байт	throughput, оп/с
A	21.8	765.9	220833	43834
B	21.0	679.2	480973	47859
C	24.0	682.8	400902	42062

Как меняется рекомендация при той же нагрузке:

Стратегия	Выбор	Почему
memory	A	минимальная память (220 KB vs 481 KB у B)
latency	B	минимальная avg latency (21.0 мкс)
p99	B	минимальный p99 (679.2 мкс)

throughput	B	максимальный throughput
balanced	B	компромисс: B лучше по latency/p99/throughput
latency-critical	B	доминирует латентность
constrained (memory, p99 ≤ 5000 мкс)	A	все проходят p99; min memory = A

Главный тезис: оптимальный подход не существует абсолютно. При одной и той же конфигурации нагрузки изменение приоритетов меняет итоговую архитектуру:

- `--strategy memory` → **Approach A** (220 833 байт против 480 973 у B).
- `--strategy latency`, `--strategy throughput` или `--strategy balanced` → **Approach B**.

Профиль нагрузки диктует физические характеристики системы (что дает ML), а бизнес-приоритеты заказчика определяют, чем можно пожертвовать (что дает стратегия). Dev 5 реализует вторую половину этой формулы, обеспечивая прозрачный, быстрый и детерминированный выбор архитектуры файловой системы.

Воспроизведение: команды запуска

Рекомендатель (§6 T3)

Из корня репозитория:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r recommender/requirements.txt

# если нет model.pkl:
python ML-Model/ModelLearner.py

python recommender/recommend.py \
  --D 10 --W 100 --F 0.6 \
  --p-read 0.6 --p-write 0.2 --p-mkdir 0.05 --p-ls 0.05 \
  --p-mv 0.03 --p-find 0.07 \
  --dist zipf --zipf-s 1.5 --locality 0.3 \
  --strategy balanced
```

Флаг `--json` выводит результат в машиночитаемом формате. Пример со стратегией ограничений:

```
python recommender/recommend.py \
  --D 10 --W 100 --F 0.6 \
  --p-read 0.6 --p-write 0.2 --p-mkdir 0.05 --p-ls 0.05 \
  --p-mv 0.03 --p-find 0.07 \
  --dist zipf --zipf-s 1.5 --locality 0.3 \
  --strategy constrained --objective memory --max-p99 5000
```

Пайплайн (§8.1 T3)

```
# бенчмарк -> recommender/data/results.csv (ML-Model не трогается)
python recommender/pipeline.py

# полный цикл: установить датасет в ML-Model/ и переобучить модель
python recommender/pipeline.py --install-dataset --train
```

Ограничения и границы применимости

Область валидности ML-предсказаний

Модель обучена на сетке: $D \in \{2, 5, 10, 15\}$, $W \in \{10, 30, 100, 300\}$, $F \in \{0.3, 0.6, 0.95\}$. Для параметров **сильно за пределами** сетки предсказания — экстраполяция, доверие к рекомендации снижается.

Зависимость от Dev 4

Качество рекомендации ограничено качеством ML-модели (R^2 , MAE — зона Dev 4). Dev 5 не переобучает модель без явного `--train` в пайплайне.

Выводы Dev 5

В рамках зоны Dev 5 реализовано:

1. **Модуль стратегий** (`strategies.py`) — 9 стратегий, включая уникальную `constrained`.
2. **Программа-рекомендатель** (`recommend.py`) — CLI по §6 с текстовым и JSON-выводом.
3. **Адаптер предсказаний** (`predict.py`) — интеграция с Dev 4 без правок ML-Model/.
4. **Пайплайн воспроизведения** (`pipeline.py`) — автоматизация §8.1 с безопасным режимом по умолчанию.
5. **Unit-тесты** — 20 шт. со слойным мокированием ML.
6. **Документация** — `recommender/README.md`, корневой `README.md`.

Подытожив: при фиксированной нагрузке смена стратегии меняет рекомендуемый подход ($A \leftrightarrow B$), что подтверждает корректность разделения «предсказание метрик» (Dev 4) и «выбор по приоритетам» (Dev 5).

Итоговые выводы команды (ТЗ §8.2)

Ниже — сводка обязательных пунктов исследования, разложенных по разделам отчёта Dev 1–5.

Проблема: невозможность полного перебора пространства параметров

Пространство конфигураций многомерно: D , W , F , профиль операций, распределение доступа, локальность, подход A/B/C. Полный перебор только профилей вероятностей (шаг 0.1, сумма = 1) даёт > 3 млн точек и $> 10^{11}$ операций бенчмарка — практически невыполнимо.

Решение команды: двухэтапный pipeline:

1. **Этап 1 (Dev 3):** измерения только в узлах заранее выбранной сетки (864 конфигурации $\times$ 3 подхода, см. раздел «Отчёт по разработке бенчмарка»).
2. **Этап 2 (Dev 4):** Random Forest аппроксимирует метрики между узлами сетки.
3. **Этап 3 (Dev 5):** рекомендатель выбирает A/B/C по бизнес-стратегии поверх предсказаний ML.

Такой подход — единственный разумный способ покрыть многомерное пространство за приемлемое время (см. «Проблема комбинаторного взрыва» в разделе ML-Model).

Корреляция метрик: p99 и средняя латентность

Анализ корреляции (раздел ML-Model, «Анализ корреляции между метриками») показал:

1. $\text{Pearson}(\text{avg_latency}, \text{p99}) = 0.0639$ — **линейная** зависимость почти не видна.
2. $\text{Spearman} = 0.9517$ — **монотонная** связь сильная, но **нелинейная**.

Вывод: p99 нельзя вычислять как $a * \text{avg_latency} + b$; нужна отдельная модель для p99. Это обосновывает четыре независимые регрессии в Dev 4.

Оптимальность подходов по областям пространства параметров

На основе асимптотики (раздел «Интерфейс ФС»), результатов бенчмарка и ML-предсказаний:

Область / профиль нагрузки	Типичный лидер	Почему	Когда проигрывает
Read-heavy (static_web, высокий P_{read})	В или С	доступ по полному пути за $O(1)$	мало RAM $\rightarrow$ А
Write/mkdir-heavy, умеренные D, W	В	быстрый lookup + дерево для ls	жёсткий лимит RAM $\rightarrow$ А
Memory-critical, embedded, много инстансов	А	минимум служебных структур и hash-overhead	нужен max throughput $\rightarrow$ В
Metadata-heavy: частый ls, find	А или В	дерево хранит детей явно; В — быстрый старт find	очень широкие директории без индекса $\rightarrow$ С медленнее
Move-heavy (refactoring, большие D, W)	А	перемещение узла в дереве дешевле, чем перестройка всех путей в map	—
Плоское сканирование без иерархии (ls/find на С)	—	—	С: полный scan hash map за $O(N)$

Важно: «типичный лидер» — не абсолютное правило, а доминирующий тренд на сетке. Итоговый выбор для конкретной точки ($D, W, F, \text{profile}, \dots$) даёт ML + стратегия рекомендателя.

Влияние стратегии выбора (Dev 5)

Стратегии определены **самостоятельно** в recommender/strategies.py (9 вариантов, см. раздел «Стратегии выбора») и обоснованы инженерными приоритетами:

1. однометричные — один жёсткий KPI;
2. взвешенные — компромисс (min-max по A/B/C);
3. constrained — SLA + вторичная цель.

Демонстрация ($D = 10, W = 100, F = 0.6, \text{zipf}$): одни и те же предсказания ML, но:

Приоритет заказчика	Стратегия	Рекомендация
только память	memory	А
только латентность / p99	latency / p99	В
throughput / баланс	throughput / balanced	В
SLA по p99 + min memory	constrained	А

Связка компонентов проекта

Задача ТЗ	Кто закрыл	Где в отчёте
Реализации A/B/C	Dev 1–2	«Реализация А», «Реализация В и С»
Бенчмарк и сетка	Dev 3	«Отчёт по разработке бенчмарка»
ML-аппроксимация, корреляция метрик	Dev 4	«ML-Model»
Рекомендатель, стратегии, пайплайн	Dev 5	«Программа-рекомендатель»

Главный вывод проекта: оптимальный подход **не существует в абсолют**. Он зависит от (1) профиля нагрузки и структуры ФС — через ML-предсказания, и от (2) приоритетов заказчика — через стратегию в рекомендателе. Программа-рекомендатель замыкает цепочку «параметры нагрузки → метрики → решение» без повторного запуска бенчмарка.

Тестирование

Каталог `tests/` — единая точка автоматической проверки проекта. Тесты собираются через CMake (`tests/CMakeLists.txt`) и делятся на **три независимых блока**: ядро файловых систем (C++/GoogleTest), бенчмарк (C++/GoogleTest) и рекомендатель (Python/unittest).

Как запускать

Из корня репозитория после конфигурации CMake:

```
mkdir -p build && cd build
cmake ..
cmake --build .
```

```
# все C++-наборы + Python-тесты recommender через CTest
ctest --output-on-failure
```

```
# или по отдельности:
```

```
./fs_core_tests
./benchmark_tests
python3 -m unittest discover -s tests/recommender -p 'test_*.py' -v
```

Python-тесты рекомендателя также подхватываются CTest (`recommender_tests`) с `WORKING_DIRECTORY` = корень репозитория.

Общая стратегия

- Изоляция зон ответственности:** каждый блок тестирует свой слой, не тянет чужие тяжёлые зависимости.
- Параметризация Case A/B/C:** тесты ядра ФС (`fs_core/`) прогоняются через GoogleTest `TYPED_TEST` для всех трёх реализаций — один сценарий проверяется на `Case_A`, `Case_B`, `Case_C`.
- Заглушки вместо реальных ФС в бенчмарке:** `FakeFileSystem` считает вызовы операций и возвращает фиксированную «память» — `runner/generator` тестируются без линковки `case_A/B/C`.
- Моки ML в recommender:** стратегии и CLI проверяются на фикстурах `SAMPLE_PREDICTIONS`, без `model.pkl` и `sklearn`.

Блок 1: `tests/fs_core/` — ядро файловой системы

Target CMake: `fs_core_tests` · Фреймворк: GoogleTest · Зона: Dev 1–2

Линкуется с реализациями case_A, case_B, case_C. Проверяет контракт IFileSystem на реальных in-memory ФС.

Файл	Что проверяется	Примеры сценариев
test_tree_structure.cpp	Структура дерева	создание веток и листьев, смена родителя
test_crud_operations.cpp	CRUD + ls/mv/find	read/write/mkdir, list, move, glob-find, коллизии
test_metrics_tracking.cpp	Счётчики и NodeState	рост file_count/dir_count, размер узла без данных файла
test_system_state.cpp	SystemState, байты файлов	накопление total_file_bytes, overwrite, move без смены счётчиков
test_edge_cases.cpp	Граничные и невалидные пути	отказ ./.., двойной /, deep/wide tree, Clear

Ключевые проверки: абсолютные пути only; корень / защищён; find с wildcard; перемещение поддерева; отказ записи поверх директории.

Блок 2: tests/benchmark/ — инфраструктура бенчмарка

Target CMake: benchmark_tests · **Фреймворк:** GoogleTest · **Зона:** Dev 3

Линкует WorkloadGenerator.cpp, BenchmarkRunner.cpp, CsvWriter.cpp. Реальные Case A/B/C не нужны — runner работает с FakeFileSystem.

Файл	Компонент	Что проверяется
test_workload_config.cpp	WorkloadConfig	сумма вероятностей = 1, валидность D/W/F, dist, zipf_s, locality, ops/repeats
test_workload_generator.cpp	WorkloadGenerator	число measured-ops, непустой setup, валидные пути, детерминизм по seed, исполнимость на FakeFS
test_csv_writer.cpp	CsvWriter	заголовок CSV, число колонок, порядок полей approach/D/W/F/метрики
test_benchmark_runner.cpp	BenchmarkRunner	пустой прогон, setup применяет операции, dispatch read/write/..., memory из SystemState

Заглушка fake_file_system.hpp: счётчики вызовов по типам операций; SystemState().memory_bytes = 4242 — достаточно для проверки агрегации метрик без полного бенчмарка.

Блок 3: tests/recommender/

Target CMake: recommender_tests · **Фреймворк:** Python unittest · **20 тестов**, <5 мс

Общие фикстуры — fixtures.py: SAMPLE_PREDICTIONS (A/B/C с четырьмя метриками), SAMPLE_PARAMS.

Файл	Модуль	Сценарии (15 + 2 + 3)
------	--------	-----------------------

test_strategies.py	strategies.py	15 тестов: 4 одномеричные, 3 weighted, equal penalties, constrained feasible/infeasible/min-throughput, unknown/empty/missing metric, список стратегий
test_predict.py	predict.py	2 теста: только predictions из Interface; ошибка если нет Interface.py
test_recommend_cli.py	recommend.py CLI	3 теста: текстовый вывод, JSON-структура, constrained через argparse

Почему мокаем ML, но не стратегии:

1. Dev 4 (model.pkl, sklearn) — вне зоны unit-тестов Dev 5; CI/локальный прогон не должен требовать 129 MB артефактов.
2. Dev 5 (strategies.py) — тестируется на **детерминированных** числах без моков: это собственная бизнес-логика.

Что сознательно не покрыто unit-тестами

1. Полный прогон бенчмарка на 864 конфигурации (интеграционный / ночной запуск).
2. Качество ML-модели (R^2 , MAE) — оценивается офлайн в Dev 4 на hold-out 20%.
3. Сквозной E2E «smake → benchmark → train → recommend» — вынесен в ручной сценарий pipeline.py и демо в отчёте.

Связь тестов с компонентами проекта

Компонент репозитория	Тестовый блок	Тип
case_A/B/C, IFileSystem	fs_core_tests	интеграционные, 3× параметризация
benchmark/*	benchmark_tests	unit + fake FS
recommender/*	recommender_tests	unit, мок ML